

4. Cuestionario de Análisis Conceptual

María Camila Salamanca Cortés A00411234

Heidy Yuliana Hernandez Huila A00411232

Responda con rigor técnico las siguientes preguntas en su informe de entrega:

1. **Delimitación de Mensajes en TCP:** TCP es un protocolo de transporte orientado a flujo de bytes continuo y no a bloques. ¿Por qué se produciría un fallo de lectura o bloqueo indefinido si el cliente enviara el JSON sin el salto de línea (\n) o sin invocar `writer.flush()`? Explique el concepto de *TCP Framing*.

R/ TCP es un protocolo de transporte orientado a un flujo de bytes continuo, no a bloques o mensajes discretos. Esto significa que no conoce el concepto de mensajes individuales a nivel de aplicación; para TCP, los datos son una secuencia ininterrumpida de bytes sin fronteras automáticas.

Si el cliente envía el JSON sin el salto de línea (\n), se produce un bloqueo indefinido porque métodos de lectura como `BufferedReader.readLine()` leen caracteres hasta encontrar un delimitador. Al no existir ese delimitador, el servidor queda esperando indefinidamente el carácter que nunca llega, sin poder procesar el mensaje.

Si el cliente no invoca `writer.flush()`, los datos se quedan almacenados temporalmente en el búfer de memoria del flujo de salida y nunca se transmiten físicamente por el socket. El servidor, por tanto, nunca recibe el JSON y permanece bloqueado esperando datos que no se envían.

El TCP Framing es la técnica de estructuración utilizada para definir los límites claros de cada mensaje dentro de ese flujo continuo de bytes. Se implementa mediante delimitadores específicos (como el carácter de nueva línea) que permiten a la aplicación receptora saber dónde termina un mensaje y comienza el siguiente.

2. **Ventajas y Sobrecargas de ThreadPools:** El servidor utiliza

`Executors.newFixedThreadPool(5)`. ¿Qué ventajas ofrece esta estrategia frente al esquema ingenuo de instanciar un `new Thread()` por cada cliente? ¿Qué sucede cuando se conectan 6 o más clientes simultáneos?

R/ El esquema ingenuo de instanciar un `new Thread()` por cada cliente consume memoria RAM, ya que cada hilo asigna su propio *stack*, y genera una sobrecarga alta de CPU debido al cambio de contexto. Este cambio de contexto se produce porque el planificador del sistema operativo (*scheduler*) se ve forzado a interrumpir de forma constante la ejecución de un hilo, guardar su contexto (registros del procesador y estado de la pila) y cargar el de otro, lo que ocurre masivamente al instanciar un hilo por cliente.

La estrategia `Executors.newFixedThreadPool(5)` limita el número máximo de hilos concurrentes activos a 5 y los reutiliza mediante una piscina (*pool*), evitando la creación y destrucción constante de hilos.

Cuando se conectan 6 o más clientes simultáneos, los primeros 5 son atendidos de inmediato y el 6º en adelante se coloca automáticamente en una cola de espera interna (`LinkedBlockingQueue`) hasta que un hilo quede libre.

3. **Condiciones de Carrera y Exclusión Mutua:** Si el servidor maneja una única instancia compartida de `BoardGame`, ¿qué problema específico de concurrencia ocurriría si dos clientes ejecutan `selectCell` en celdas contiguas exactamente al mismo milisegundo sin métodos `synchronized`?

R/

Una *condición de carrera* (*race condition*) ocurre cuando dos o más hilos acceden y modifican un recurso compartido de forma concurrente, y el resultado final depende del orden exacto de ejecución del procesador.

Si dos clientes ejecutan `selectCell` en celdas contiguas exactamente al mismo milisegundo sin métodos `synchronized`, al modificar la instancia compartida `BoardGame` simultáneamente se produce un conflicto de lectura-escritura. Ambos hilos podrían leer el tablero como disponible al mismo

tiempo y actualizar la matriz a la vez, provocando la sobrescritura de datos o un estado completamente inconsistente. El uso de `synchronized` garantiza la exclusión mutua y evita este problema.

4. **Conexiones Cortas vs. Conexiones Persistentes:** En el diseño actual, el servidor cierra el socket tras responder cada comando (*Short-lived connection*). Compare este enfoque frente a mantener el socket abierto durante toda la partida (*Persistent connection*) en términos de consumo de descriptores de archivo, latencia y escalabilidad.

R/

En el diseño de **conexiones cortas** (*Short-lived connection*), el cliente abre un socket, envía un comando, el servidor responde y el socket se cierra inmediatamente. Esto implica que los descriptores de archivo se abren y cierran masivamente, acumulando conexiones en estado `TIME_WAIT`. La latencia es alta por el apretón de manos de TCP repetitivo, y la escalabilidad se degrada severamente bajo alta frecuencia de peticiones.

En el diseño de **conexiones persistentes** (*Persistent connection*), el socket se mantiene abierto durante toda la sesión de la partida. Se mantiene un único descriptor abierto por cliente activo, por lo que se requiere controlar adecuadamente los límites del sistema operativo (como el parámetro `ulimit`) para evitar restricciones o rechazos del kernel al superar el número máximo permitido de archivos/sockets abiertos por proceso. La latencia es mínima al estar el canal previamente establecido, y la escalabilidad es excelente para tiempo real, exigiendo una eficiente gestión de conexiones inactivas.

5. **Declaración y Reflexión sobre IAG:** Declare explícitamente qué herramientas de Inteligencia Artificial Generativa utilizó durante el desarrollo del laboratorio, qué prompts empleó para formular la solución del cliente o depurar el servidor, y cómo validó que el código propuesto por la IA cumpliera las reglas del protocolo sin introducir vulnerabilidades.

R/ Declaración y Reflexión sobre IAG

- **Herramientas utilizadas:** Asistentes de Inteligencia Artificial Generativa empleados como apoyo en la consulta conceptual, diseño de estructuras de sockets y depuración de errores lógicos o de concurrencia.
- **Prompts empleados:** Consultas orientadas a resolver problemas específicos, por ejemplo: *"Explica cómo estructurar la lectura de sockets en Java usando BufferedReader para evitar bloqueos por falta de delimitadores"* o *"Ejemplo de cómo aplicar bloques synchronized para prevenir condiciones de carrera en objetos de juego compartidos"*.
- **Validación de la solución:** El proceso de validación combinó una rigurosa lectura e información previa sobre las reglas del protocolo con experimentación empírica mediante prueba y error (*trial and error*) para verificar el comportamiento operativo del sistema. Asimismo, se consultó activamente a la IA sobre las posibles vulnerabilidades del código y las razones técnicas de su existencia, lo que permitió detectar fallos ocultos, auditar la seguridad de la implementación y corregir preventivamente debilidades antes de la entrega final.

CAPTURAS

```
MainClient.java x Run: MainClient x
0 - - - - -
1 - - - - -
2 - - - - -
3 - - - - -
4 - - - - -

--- MENÚ DE ACCIONES ---
1. Iniciar nuevo juego (INIT_GAME)
2. Obtener tablero actual (GET_BOARD)
3. Seleccionar casilla (SELECT_CELL)
4. Marcar/Desmarcar bandera (MARK_CELL)
5. Revelar todo (SOW_ALL)
6. Salir
Elige una opción:

0 1 2 3 4
0 - - - - -
1 - - - - -
2 - - - - -
3 - - - - -
4 - - - - -

--- MENÚ DE ACCIONES ---
1. Iniciar nuevo juego (INIT_GAME)
2. Obtener tablero actual (GET_BOARD)
3. Seleccionar casilla (SELECT_CELL)
4. Marcar/Desmarcar bandera (MARK_CELL)
5. Revelar todo (SOW_ALL)
6. Salir

Problems 1 Output Debug Console Terminal Ports

2 [ . . . . . ]
3 [ . . . . . ]
4 [ . . . . . ]
Client disconnected: /127.0.0.1
Client connected: /127.0.0.1

0 1 2 3 4
0 [ . . . . . ]
1 [ . . . . . ]
2 [ . . . . . ]
3 [ . . . . . ]
4 [ . . . . . ]
Client disconnected: /127.0.0.1
Client connected: /127.0.0.1

0 1 2 3 4
0 [ . . . . . ]
1 [ . . . . . ]
2 [ . . . . . ]
3 [ . . . . . ]
4 [ . . . . . ]
Client disconnected: /127.0.0.1
[]
```

```
MainClient.java x Run: MainClient x Run: MainClient x
[Servidor] Estado: OK

--- TABLERO ---
0 1 2 3 4
0 - - - - -
1 - - - - -
2 - - - - -
3 - - - 2 -
4 - - - - -

--- MENÚ DE ACCIONES ---
1. Iniciar nuevo juego (INIT_GAME)
2. Obtener tablero actual (GET_BOARD)
3. Seleccionar casilla (SELECT_CELL)

[Servidor] Estado: OK

--- TABLERO ---
0 1 2 3 4
0 - - - - -
1 - - - - -
2 - - - - -
3 - - - 2 -
4 - - - - -

--- MENÚ DE ACCIONES ---
1. Iniciar nuevo juego (INIT_GAME)
2. Obtener tablero actual (GET_BOARD)
3. Seleccionar casilla (SELECT_CELL)

Problems 1 Output Debug Console Terminal Ports Run: MainClient

2 [ . . . . . ]
3 [ . . . . . ]
4 [ . . . . . ]
Client disconnected: /127.0.0.1
Client connected: /127.0.0.1

0 1 2 3 4
0 [ . . . . . ]
1 [ . . . . . ]
2 [ . . . . . ]
3 [ . . . 2 . ]
4 [ . . . . . ]
Client disconnected: /127.0.0.1
Client connected: /127.0.0.1

0 1 2 3 4
0 [ . . . . . ]
1 [ . . . . . ]
2 [ . . . . . ]
3 [ . . . 2 . ]
4 [ . . . . . ]
Client disconnected: /127.0.0.1
[]
```

Elige una opción: 5

[Servidor] Estado: OK

--- TABLERO ---

```
  0 1 2 3 4
0  0 1 * 1 0
1  1 3 3 2 0
2  2 * * 2 0
3  2 * * 2 0
4  1 2 2 1 0
```

--- MENÚ DE ACCIONES ---

1. Iniciar nuevo juego (INIT_GAME)

Elige una opción: 4

Fila (i): 3

Columna (j): 3

[Servidor] Estado: OK

--- TABLERO ---

```
  0 1 2 3 4
0  0 1 * 1 0
1  1 3 3 2 0
2  2 * * 2 0
3  2 * * 2 0
4  1 2 2 1 0
```

Problems 1 Output Debug Console Terminal Ports

```
2 [ . . . . . ]
3 [ . . . 2 . ]
4 [ . . . . . ]
Client disconnected: /127.0.0.1
Client connected: /127.0.0.1
```

```
  0 1 2 3 4
0 [ 0 1 * 1 0 ]
1 [ 1 3 3 2 0 ]
2 [ 2 * * 2 0 ]
3 [ 2 * * 2 0 ]
4 [ 1 2 2 1 0 ]
Client disconnected: /127.0.0.1
Client connected: /127.0.0.1
```

```
  0 1 2 3 4
0 [ 0 1 * 1 0 ]
1 [ 1 3 3 2 0 ]
2 [ 2 * * 2 0 ]
3 [ 2 * * M 0 ]
4 [ 1 2 2 1 0 ]
Client disconnected: /127.0.0.1
```

□